

Registro de Decisiones de Arquitectura (ADR)

ADR 001: Ingesta de Datos IoT mediante MQTT y Suscripciones Compartidas

Contexto:

El dispositivo de hardware (XIAO ESP32S3) necesita transmitir ráfagas de datos críticos en tiempo real, incluyendo telemetría (latitud, longitud, tipo de emergencia), imágenes capturadas desde la PSRAM y fragmentos (*chunks*) de audio PDM. Se requiere un protocolo ligero que soporte conexiones inestables y permita la distribución de la carga de trabajo entre múltiples instancias del servicio de recepción, sin que un solo nodo se sature procesando la misma alerta.

Alternativas Consideradas:

1. **HTTP/REST:** Descartado. Genera demasiado *overhead* en los encabezados para el ESP32, no es bidireccional por naturaleza y la transferencia de audio por *chunks* requeriría múltiples conexiones TCP costosas.
2. **WebSockets:** Descartado. Aunque mantiene una conexión persistente, la gestión de balanceo de carga para archivos binarios masivos desde múltiples ESP32 es compleja.
3. **MQTT con Eclipse Mosquitto:** Seleccionado.

Decisión:

Se implementó **MQTT** como protocolo de ingesta principal, utilizando **Suscripciones Compartidas** (\$share/grupo_c5/c5/alerts) en el clúster de microservicios de recepción (reception1, reception2, reception3). El dispositivo publica los datos binarios en tópicos específicos, dividiendo el audio en fragmentos de 4096 bytes.

Justificación:

MQTT opera bajo un modelo publicador/suscriptor ideal para ambientes IoT con ancho de banda limitado. La decisión de usar el prefijo \$share es arquitectónicamente crucial: permite que Mosquitto funcione como un balanceador de carga nativo (*Round-Robin*). Cuando una alerta entra al sistema, solo **una** instancia de la flota de recepción la procesa. Esto se complementa con un bloqueo

distribuido en Redis (lock:device:{device_id}) durante 20 segundos, asegurando que todos los *chunks* de audio y fotografías subsecuentes de ese dispositivo sean ruteados y ensamblados por el mismo trabajador temporal, evitando condiciones de carrera y pérdida de integridad en la evidencia de la alerta.

ADR 002: Comunicación Síncrona Inter-Servicios mediante gRPC

Contexto:

Una vez que el servicio de recepción ensambla la alerta, necesita evaluar inmediatamente la prioridad de la misma utilizando modelos de Inteligencia Artificial (Spanish BERT) y clasificar la zona geográfica (ej. Jardín Central de Jilotepec). Este procesamiento debe ocurrir antes de que la alerta sea encolada para persistencia y notificación, garantizando que el *dashboard* reciba información ya enriquecida.

Alternativas Consideradas:

1. **RESTful APIs (JSON sobre HTTP/1.1):** Descartado. La serialización/deserialización constante de JSON introduce latencia innecesaria. Además, los contratos no son estrictos por defecto.
2. **Mensajería Asíncrona (RabbitMQ / Redis PubSub):** Descartado para este flujo específico. La clasificación de la IA y el mapeo geoespacial son requisitos previos indispensables (*blocking operations*) para despachar la alerta; usar colas asíncronas aquí aumentaría la complejidad del flujo de estado.
3. **gRPC (Protocol Buffers sobre HTTP/2):** Seleccionado.

Decisión:

Se adoptó **gRPC** para la comunicación síncrona interna entre el servicio de recepción y los servicios de dominio intensivo (priority y geolocation). Se definieron contratos estrictos mediante archivos .proto (ej. priority.proto).

Justificación:

gRPC utiliza HTTP/2, permitiendo la multiplexación de solicitudes sobre una sola conexión TCP, y serializa los datos en formato binario (Protobuf), lo que es drásticamente más rápido y ligero que JSON. En un sistema C5 donde los milisegundos pueden representar vidas humanas, gRPC garantiza que la invocación a la IA para el análisis *zero-shot classification* y el cálculo de cuadrantes

se ejecute con latencia mínima. Además, los contratos Protobuf aseguran que cualquier cambio en la estructura de datos obligue a actualizar todos los microservicios, previniendo errores en tiempo de ejecución por disparidad de interfaces.

ADR 003: Arquitectura de Persistencia Híbrida (CQRS Simplificado) con PostgreSQL y Redis

Contexto:

El sistema presenta un clásico desafío de cargas de trabajo mixtas:

1. **Escritura intensiva:** Constante llegada de alertas, registros de auditoría de IA y actualizaciones de estado desde el ESP32.
2. **Lectura intensiva:** Múltiples operadores en el *dashboard* consultando historiales, aplicando filtros geográficos y recargando el sistema simultáneamente. Un solo motor de base de datos manejando ambas operaciones podría crear cuellos de botella y bloqueos de tablas.

Alternativas Consideradas:

1. **Base de datos única (Solo PostgreSQL Master):** Descartado. Riesgo de degradación de rendimiento en el panel de control durante picos de emergencias masivas debido al bloqueo de escritura.
2. **Base de datos NoSQL (MongoDB):** Descartado. Se requiere integridad referencial estricta y capacidades analíticas relacionales para la auditoría de las decisiones de la IA.
3. **PostgreSQL Master-Replica + Redis Queues:** Seleccionado.

Decisión:

Se diseñó un patrón de persistencia asíncrona y segregación de operaciones inspirado en CQRS (*Command Query Responsibility Segregation*). Las escrituras ("comandos") se absorben primero en listas de **Redis** (*history_queue*). Un hilo trabajador (*worker*) dedicado en el servicio history consume esta cola e inserta los datos en el nodo **PostgreSQL Maestro**. Las lecturas ("consultas") provenientes del *dashboard* mediante la API REST se dirigen exclusivamente al nodo **PostgreSQL Réplica**.

Justificación:

Esta arquitectura garantiza la Alta Disponibilidad (HA) y resiliencia del sistema C5:

- **Absorción de Picos:** Redis actúa como un "amortiguador" (*buffer*) en memoria RAM. Si hay un pico masivo de alertas, la base de datos relacional no se satura; Redis encola los eventos y el *worker* los persiste a un ritmo seguro.
- **Desacoplamiento de Cargas:** El panel de alertas lee de la Réplica, lo que significa que las consultas pesadas de los operadores nunca interferirán ni ralentizarán las escrituras críticas de nuevas emergencias que entran al Maestro.
- **Tolerancia a fallos parcial:** Si la base de datos principal experimenta lentitud momentánea, el sistema de ingesta (ESP32 -> MQTT -> Reception -> Redis) sigue funcionando ininterrumpidamente.